

Final Project

CMPT459: Data Mining, Fall 2024

Anagh Arya, 301416450

Note: As I will be working alone on this project, following the advice of the instructor, I will be implementing and analyzing one clustering and one classification algorithm.

For this project, I will be using a Bank Marketing database from the UCI repository. The data is related to direct marketing campaigns (phone calls) of a Portuguese banking institution, and can be accessed at the following:

<https://archive.ics.uci.edu/dataset/222/bank+marketing>

Problem Definition

To predict if the client will subscribe (yes/no) to a term deposit.

1. Dataset

For this project, I will be using a Bank Marketing database from the UCI repository. The data is related to direct marketing campaigns (phone calls) of a Portuguese banking institution, and can be accessed at the following:

Dataset Characteristics: The dataset consists of 45,211 instances, has 16 features and is multivariate. It is used for classification tasks and includes both categorical and integer feature types.

Feature Properties

Name	Type	Demographic	Description	Missing
age	Integer	Age	age	no
job	Categorical	Occupation	type of job (categorical: 'admin.', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self-employed', 'services', 'student', 'technician', 'unemployed', 'unknown')	no

Name	Type	Demographic	Description	Missing
marital	Categorical	Marital Status	marital status (categorical: 'divorced','married','single','unknown'; note: 'divorced' means divorced or widowed)	no
education	Categorical	Education Level	(categorical: 'basic.4y','basic.6y','basic.9y','high.school','illiterate','professional.course','university.degree','unknown')	no
default	Binary		has credit in default?	no
balance	Integer		average yearly balance	no
housing	Binary		has housing loan?	no
loan	Binary		has personal loan?	no
contact	Categorical		contact communication type (categorical: 'cellular','telephone')	no
day_of_week	Date		last contact day of the week	no
month	Date		last contact month of year (categorical: 'jan', 'feb', 'mar', ..., 'nov', 'dec')	no
duration	Integer		last contact duration, in seconds (numeric). Important note: this attribute highly affects the output target (e.g., if duration=0 then y='no'). Yet, the duration is not known before a call is performed. Also, after the end of the call y is obviously known. Thus, this input should only be included for benchmark purposes and should be discarded if the intention is to have a realistic predictive model.	no
campaign	Integer		number of contacts performed during this campaign and for this client (numeric, includes last contact)	no
pdays	Integer		number of days that passed by after the client was last contacted from a previous campaign (numeric; -1 means client was not previously contacted)	no
previous	Integer		number of contacts performed before this campaign and for this client	no
poutcome	Categorical		outcome of the previous marketing campaign (categorical: 'failure','nonexistent','success')	no
y	Binary		has the client subscribed to a term deposit?	no

2. Data Preprocessing

The code handles missing values using SimpleImputer, imputing the median and most frequent category for the numerical and categorical features respectively. For normalization we use StandardScaler to ensure a mean of 0 and variance of 1, with categorical features encoded using the OneHotEncoder, which helps avoid ordinal relationships between categories.

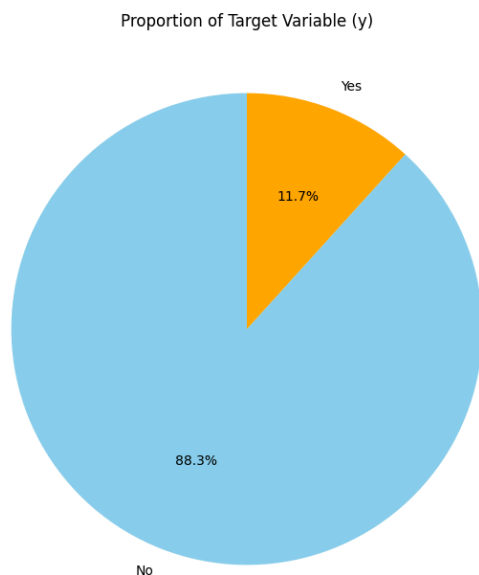
As we discuss in the next section, the classes in this dataset are heavily imbalanced. This has informed the choice of SVM as a classification algorithm in order to employ class-weight balancing to counteract this effect.

Since the dataset contains only 16 features, dimensionality reduction is not necessary. For the many scatter plots/visualizations however, PCA is used to reduce dimensionality to 2.

3. Exploratory Data Analysis (EDA)

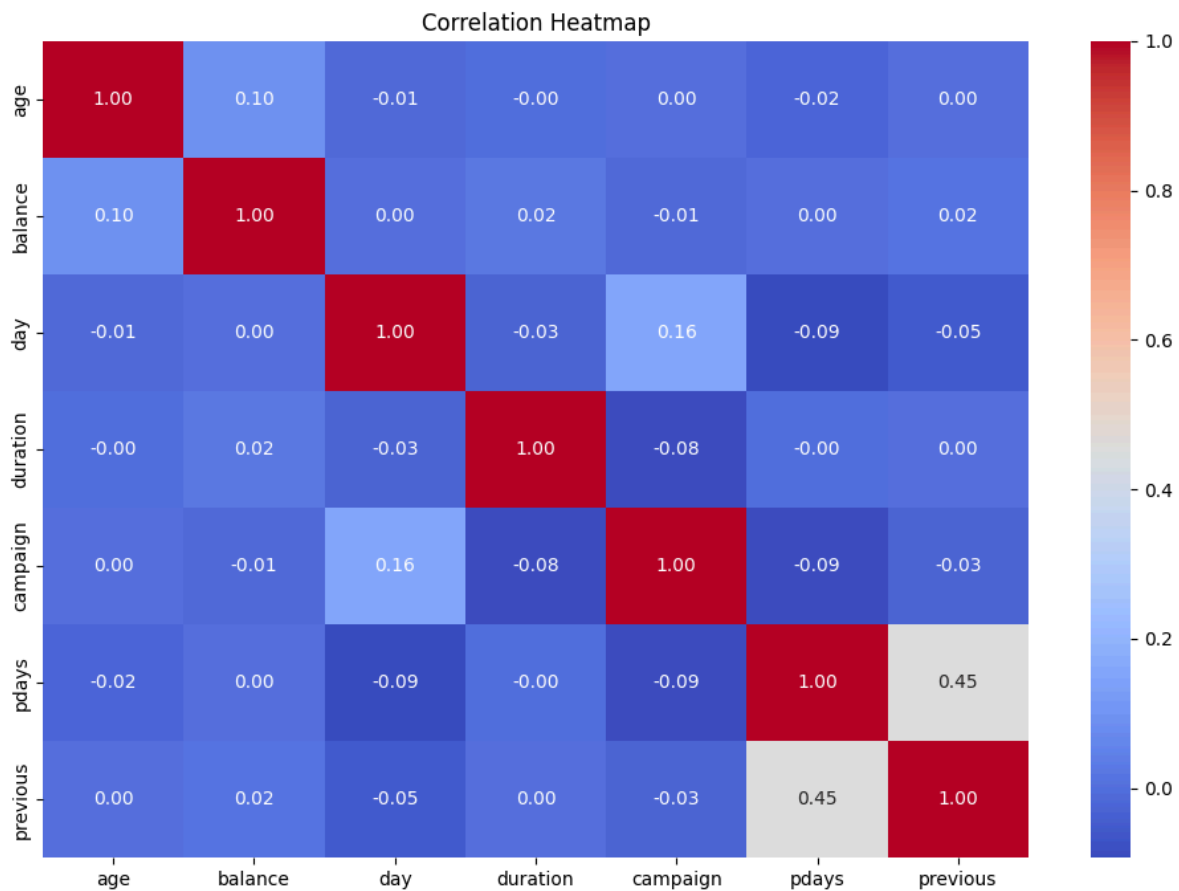
The shape of the dataset is (45211, 17).

We can gain a better idea of the class imbalance within the dataset with a plot.



We can see that the yes class only makes up 11.7% of the database, with the no class making up the other 88.3%.

To counteract this imbalance skewing the results of our classification model, we will use class weighted SVM.



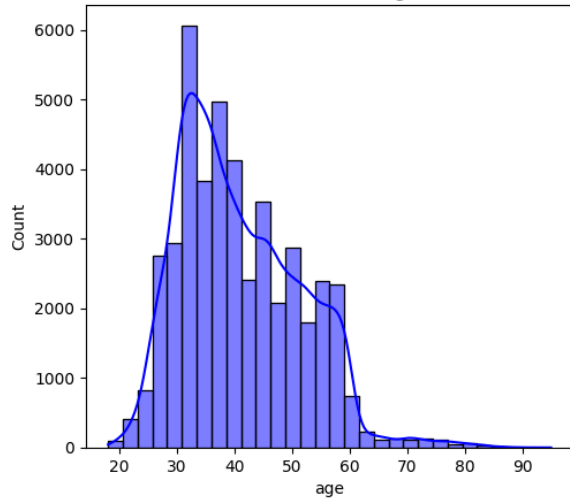
Plotting a heatmap for this dataset we observe that on average there is weak to negligible correlation between most of the features, with values very close to 0. Some features, like age and balance (0.10) and campaign and day (0.16) show a slight positive correlation.

Pairs like duration and campaign (-0.08) and balance and day (0.00) show negative or zero correlation. This means that most of the features contribute meaningful, independent information about the classification, which means careful consideration of feature selection becomes more crucial to avoid loss of information.

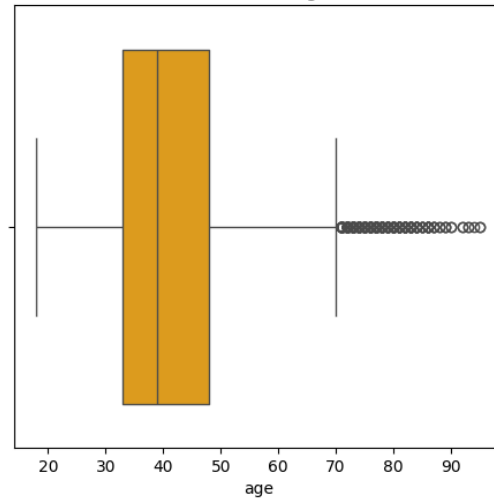
Cases like previous and pdays (0.45) are highly correlated and this may be redundant, but they are few and far between.

We can plot some of the more important features as understood in section 7 to gain a better understanding of their distributions.

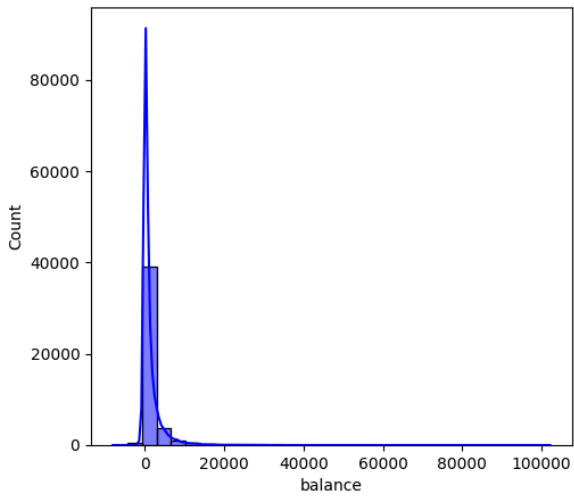
Distribution of age



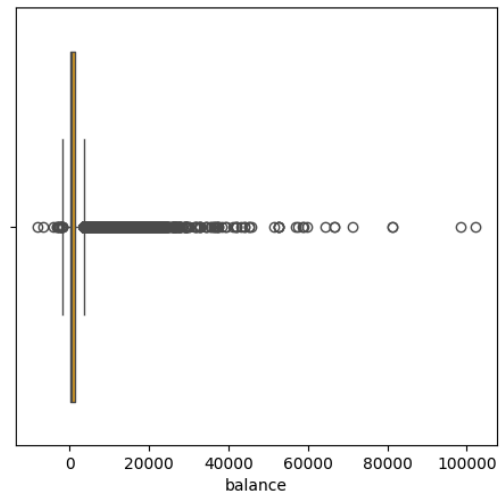
Box Plot of age



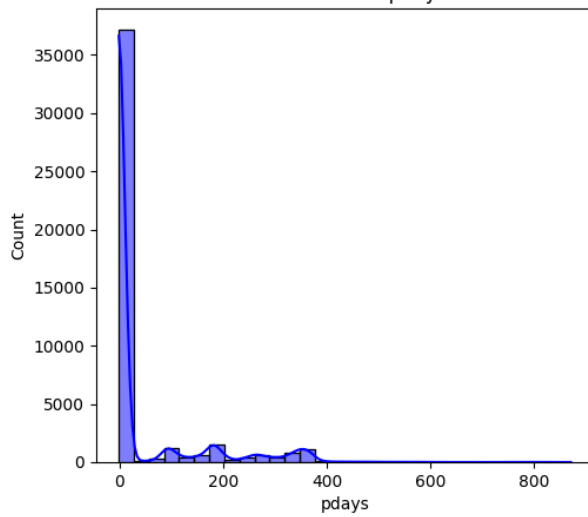
Distribution of balance



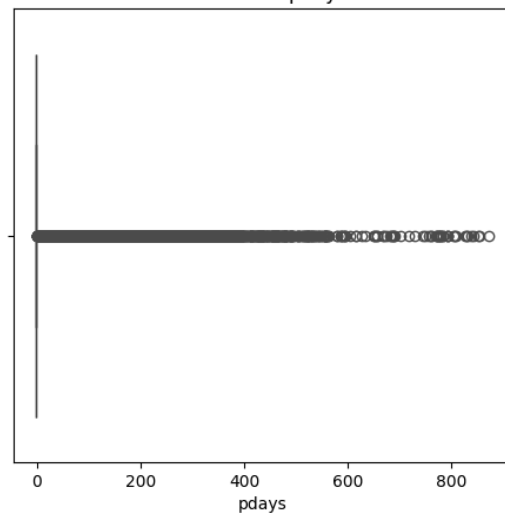
Box Plot of balance

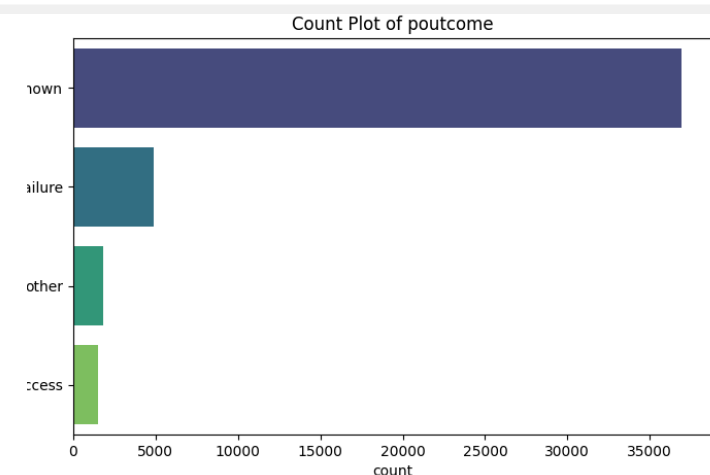
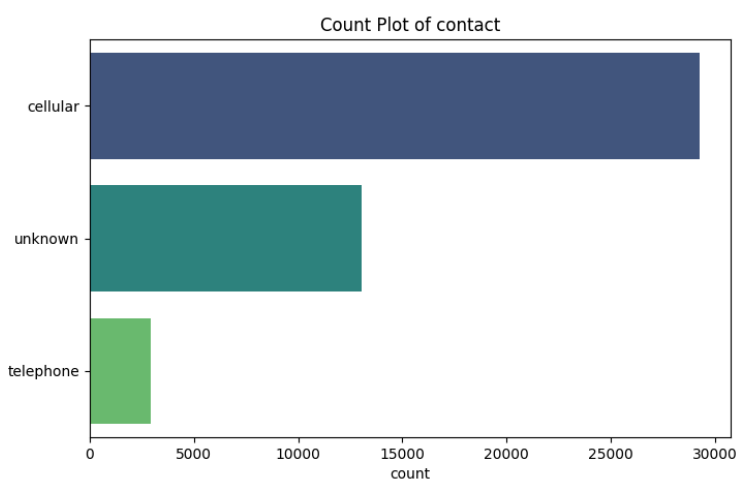
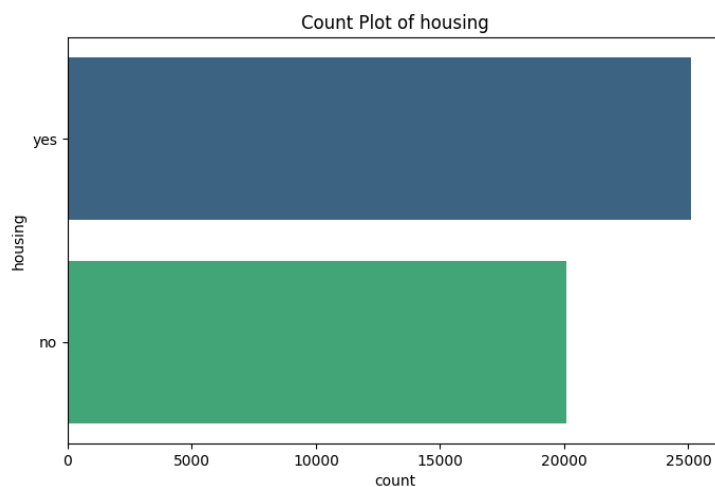
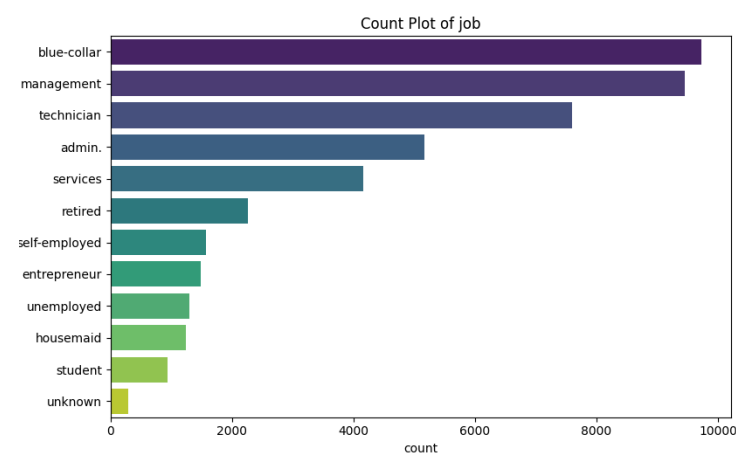


Distribution of pdays



Box Plot of pdays

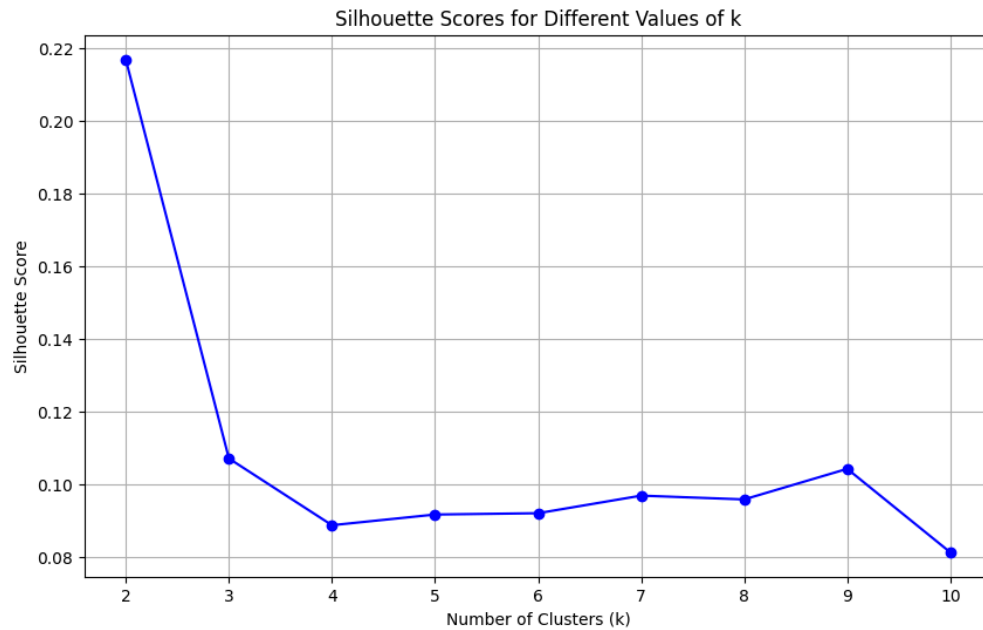




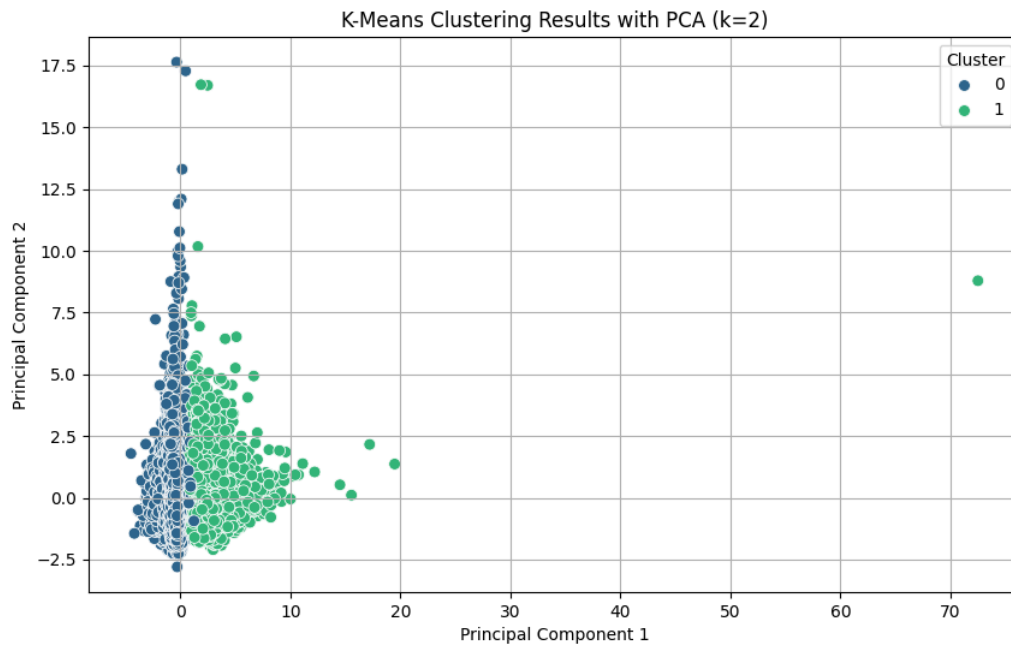
We can see that the primary demographic represented in this data are middle aged professionals in blue-collar, management and technical roles. This can inform the marketing strategy for the bank if they are reflected heavily in the target variable. We can also observe some features like balance and duration being highly skewed, with balance being prone to many outliers. The decision to either filter or transform these instances is dictated by the bank's strategy when it comes to customer outreach. Features like poutcome and contact have class imbalances that we can choose to address moving forward, along with imputation to be considered for the large number of "unknown" labels in some features depending on where we land after feature selection.

4. Clustering

We will be using the K-Means clustering algorithm for this database. The first step we take is to find the best k value for our dataset.



Plotting the Silhouette Scores against k, we see that $k = 2$ is the best fit for our database.



Scatter plot of clusters with using PCA to reduce dimensionality to 2.

As we are pursuing a binary classification for this database, a k parameter of 2 makes intuitive sense, and is reflected to be the best choice in the data with a silhouette score of 0.22. However, there are clearly some significant outliers present in the data, which we need to deal with next. To keep the sections consistent, we will discuss the results of the outlier detection and elimination here.

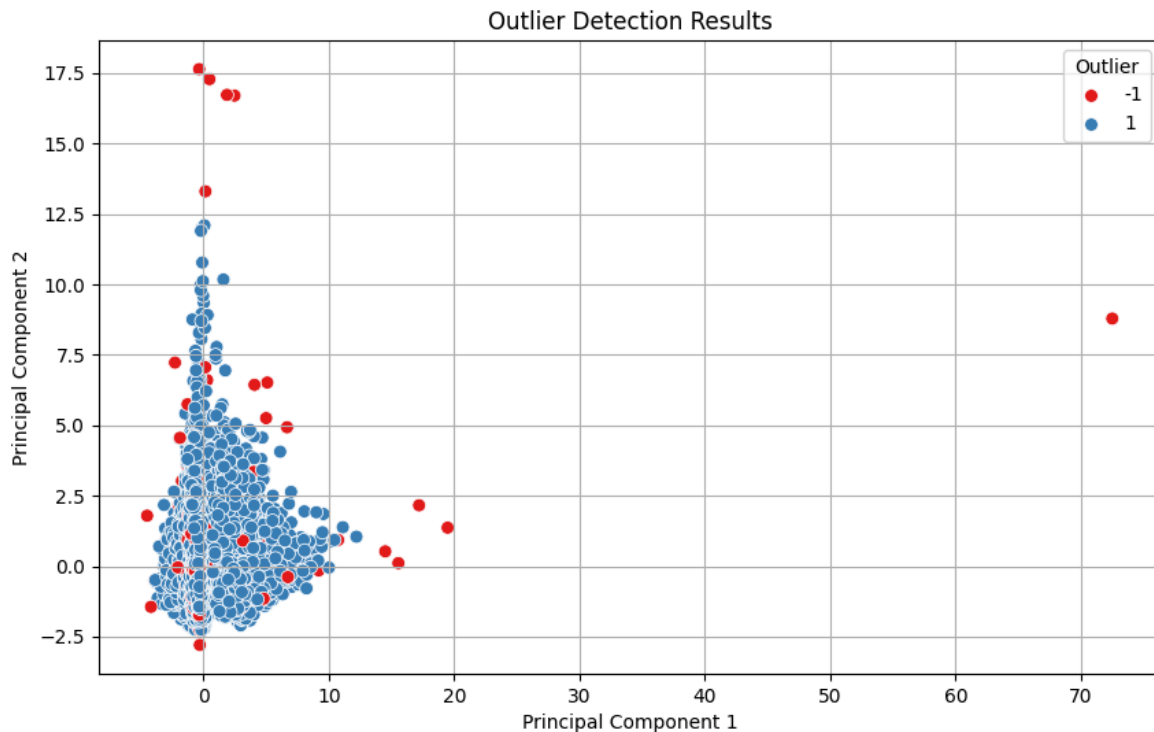
Here is what the clustering for K-Means ($k = 2$) looks like after filtering out outliers, using PCA to reduce dimensionality to 2 to produce a scatter plot.



The clustering becomes a lot more apparent without the outliers. We also see a small increase in the silhouette score (0.23) to reflect this change. The problem of class imbalance is being counteracted with using weighted classes in the SVM classifier. This explains the low silhouette score achieved by the algorithm. To get better results from clustering, we would have to use techniques like SMOTE to augment the minority class.

5. Outlier Detection

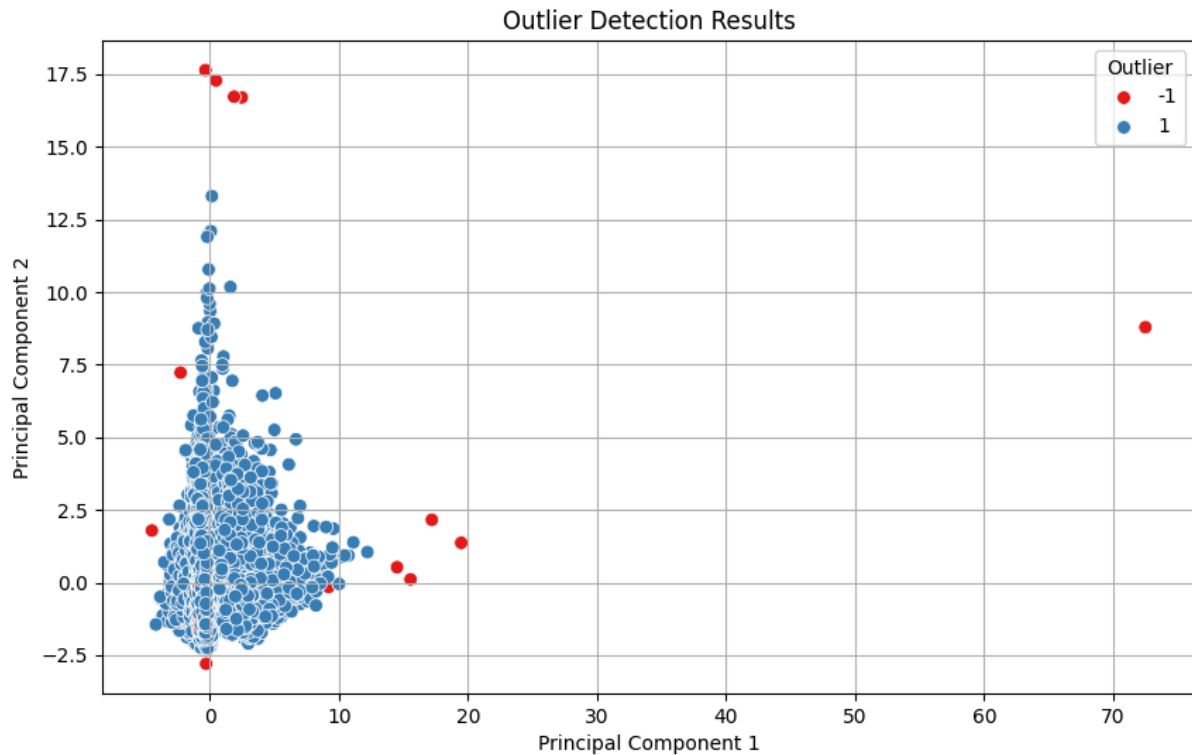
We will be using the Local Outlier Factor algorithm for this task as I found it a good fit for the dataset we are working with. The initial parameters are set to 20 for neighbours and 0.1 for contamination. Visualizing the scatter plot with PCA (2 dimensions), we get the following distribution of outliers.



The total outliers detected by the Local Outlier Factor algorithm for these parameters were 4521. Analyzing this plot, we can deduce that our function is categorizing far too much noise as outliers.

We repeat the process with a post PCA dataset, and arrive at 2261 outliers. The result is still noisy and analyzing post PCA data leads to loss of nuance in detecting outliers, so I changed my approach for the following step and returned to our pre-processed data.

Informal tuning and observation of the results illustrated a contamination of 0.01 to be a good setting. The scatter plot of outliers with this configuration is included below.



The total number of outliers produced with these parameters are 453, making up only 0.01% of our dataset. While some noise is still captured, we can feel quite confident that filtering these outliers will not reduce our information pool to any significant degree.

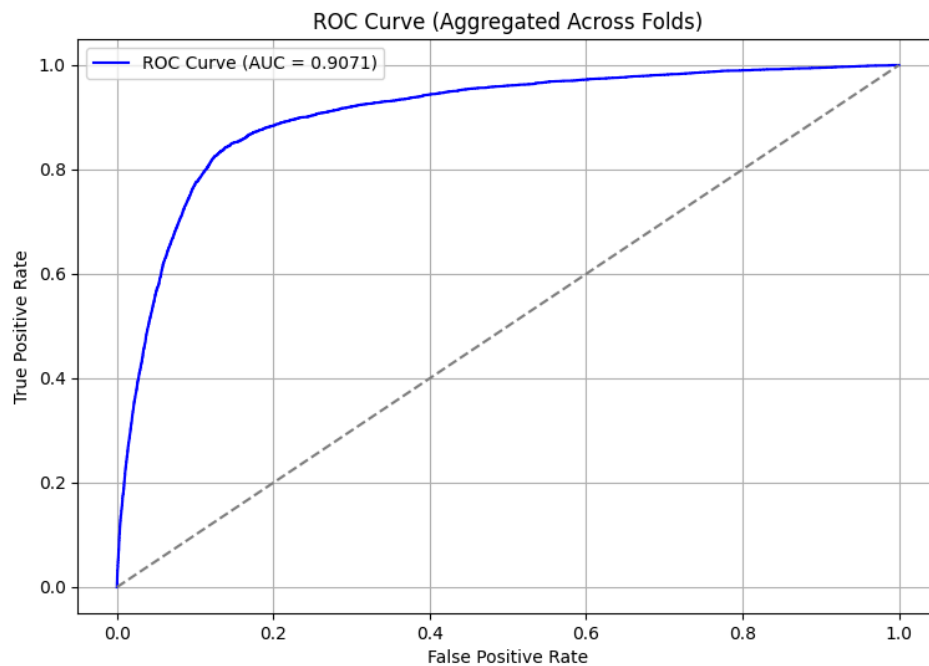
From this step onward, for the clustering, classification, feature selection and hyperparameter tuning tasks, we will be working with the dataset with the outliers filtered out.

6. Classification

For the sake of clarity, we will tackle Classification before Feature selection, as it will help us better evaluate the effects of feature selection.

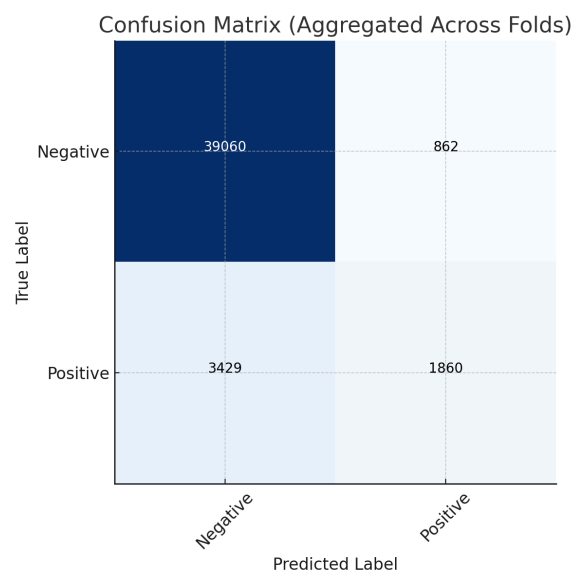
We will be using Support Vector Machines(SVM) for this classification task. We will be dividing the database 5-fold to perform cross-validation and check consistency. All values presented will be averages.

Here's the ROC curve for the classification model.



We can see that the ROC AUC is reasonably high at 90.71%. Using SVM's to balance the skewed nature of the classes helps with high accuracy.

Next, we can take a look at the confusion matrix produced.



As the database is highly skewed towards the negative, we can see the high true negative classification making up most of the task at hand.

We can understand the results better after analyzing some of the following values.

Accuracy:	0.9051
Precision:	0.6833
Recall:	0.3517
F1-Score:	0.4642
ROC-AUC:	0.9074

From the high accuracy and ROC-AUC, we can note that the overall performance of the classification model is exceptional. The precision is moderate, which can be a choice that is made by a bank institution in this case to conserve resources and only go after customers that have the highest chance of signing up for their services. The recall, however, is less than moderate. The model clearly is not fitting to the desired class as well as we could want to, and further steps need to be taken to account for the class imbalance. The F1-score represents a modest balance between the precision and recall of the model.

Here are the efficiency metrics, which we can use in the next step to compare the efficiency of the evaluation with different feature sets.

Average Training Time per Fold:	248.7563 seconds
Average Prediction Time per Fold:	20.7689 seconds

7. Feature Selection

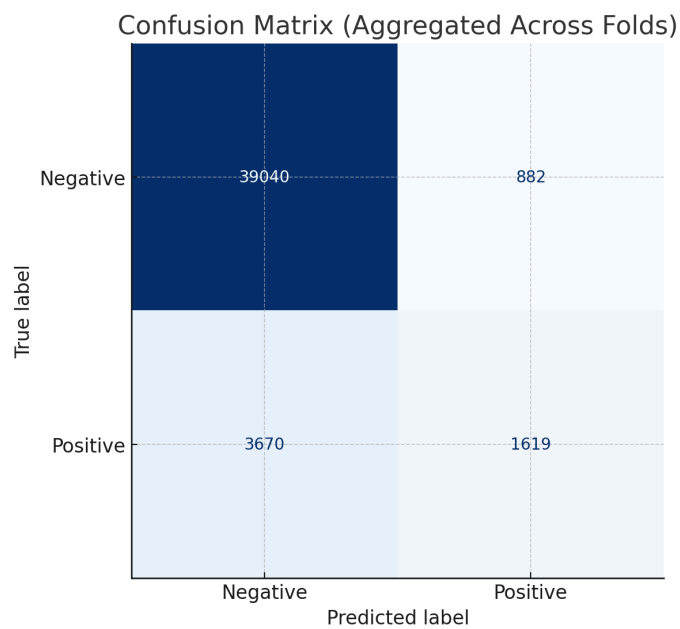
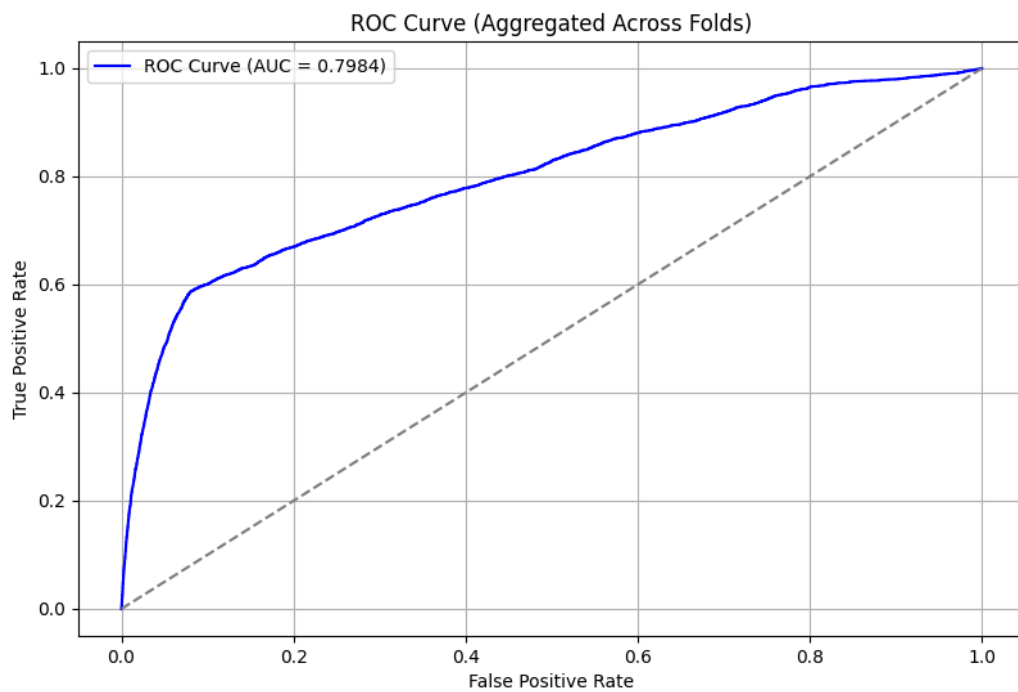
We will be using Mutual Information to select features for evaluation going forward.

Our aim is to reduce the number of features from 17 to 10, and in doing so increase the efficiency and performance of the classification while keeping a loss in accuracy to a minimum. To do this, we find the MI Scores for all features and select the top 10.

The following are the chosen features and their MI scores.

No.	Feature	Mutual Information
1	duration	0.073354
2	pdays	0.030737
3	poutcome_success	0.029916
4	balance	0.021617
5	housing_yes	0.017223
6	poutcome_unknown	0.016628
7	contact_unknown	0.016561
8	previous	0.015380
9	contact_cellular	0.014241
10	housing_no	0.012566

Upon classification using the selected features, this is the ROC curve produced.



We can observe a reduction in the ROC-AUC of almost 11% between the two datasets.

This difference is understood better with the following metrics.

Accuracy:	0.8993
Precision:	0.6472
Recall:	0.3061
F1-Score:	0.4154
ROC-AUC:	0.8076

We can see a reduction in precision, recall and F1-score of 6%, 14%, and 12% respectively. While accuracy remains mostly unchanged, the model performs worse on true values. This is a result of losing a significant portion of data by removing almost 40% of the features. The model loses a significant amount of its nuance in classifying the imbalance class as a direct result of this. However, it is far more efficient than the previous mode.

Average Training Time per Fold:	163.7361 seconds
Average Prediction Time per Fold:	18.0269 seconds

The average training and prediction times are reduced by 35% and 13% respectively. This is a massive boost to efficiency and is extremely valuable for the next step. Hyperparameter tuning is a very expensive process which can require hundreds to thousands of models being analyzed. This increased efficiency at that scale results in very significant computational and time savings. Hyperparameter tuning should also help offset the loss in accuracy that the model experiences.

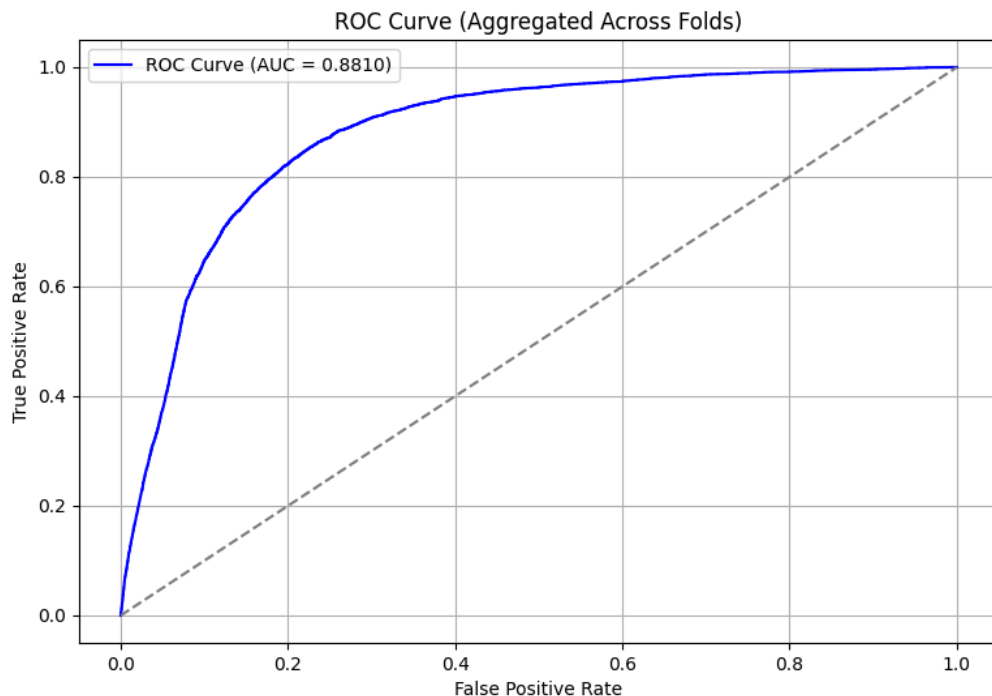
8. Hyperparameter Tuning

For tuning I chose the Bayesian optimization algorithm. I made this choice because a few of the parameters for SVM were already dictated by my EDA of the database and couldn't be changed. Dealing with such a large dataset, Random or Grid Search with cross verification produced runtimes that with parallelization often caused deadlocks and on single processors would take hours to days.

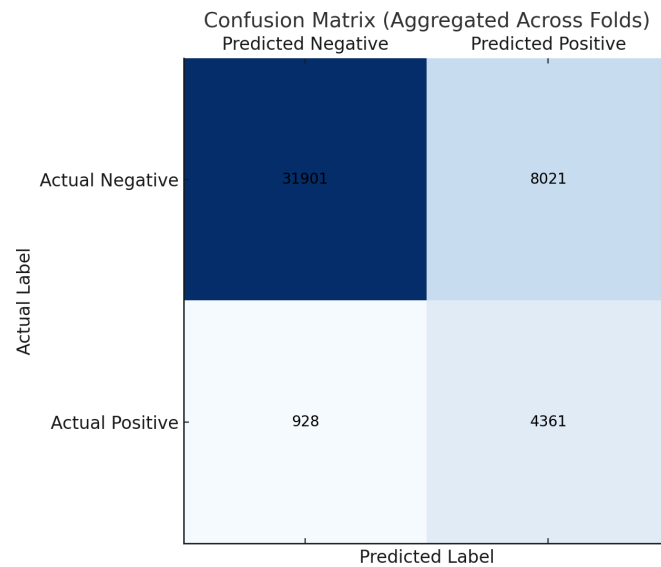
Bayesian optimization on the other hand, can rule out certain parameters as it converges on a near-optimal solution, which I found to be satisfactory for my use case.

The hyperparameter tuning process, when optimized for ROC_AUC, produced the parameters: kernel='rbf', C=2.2132, gamma='scale', class_weight='balanced'

Using these parameters to build a model achieved the following results:



We see an immediate (10%) increase in the ROC-AUC when compared to the previous model. The other changes are made more obvious when observing the confusion matrix.



Accuracy:	0.8821
Precision:	0.3524
Recall:	0.8245
F1-Score:	0.4937
ROC-AUC:	0.8812

We observe a decrease in Precision but a massive increase in Recall, more than doubling the previous model's numbers. This, of course, is extremely important in a dataset with such a heavy class imbalance where missing potential customers can be far more costly in the long run. As a result, we also see an increase in the F1-score in this model. Because of the inverse nature of these differences, the accuracy in this model remains largely the same as the previous. Regardless, this model gives us a far more useful classification tool for the banking industry as high Recall is far more important than Precision when it comes to potential customer acquisition.

Average Training Time per Fold:	241.4442 seconds
Average Prediction Time per Fold:	32.2941 seconds

These parameters do decrease the efficiency of the model, but the trade off is reasonable for a significantly more sophisticated classification model.

9. Conclusion

This analysis of the bank marketing dataset highlights several key customer behaviours. The significance of seemingly unrelated things like the contact method on the probability of placing term deposits was incredibly surprising to me. The more obviously important features proved to be past campaign outcomes, salaries, and demographics like occupation. Never having worked with a dataset with a very heavy class imbalance, this was a great learning opportunity for me. This project reinforced the importance of data preprocessing, having to deal with so many “unknown” values. I learned the implementation of feature selection algorithms and exploratory data analysis. Running a low-spec laptop, I also had to problem solve the complexity of my algorithms with tweaking parameters. This served to highlight the importance of hyperparameter tuning, which significantly helped the performance of my model in the end. I'm very happy with my dataset choice and the implementation of this data mining project. I would like to explore more datasets that deal with predicting human behaviour through data, as I am deeply fascinated by this topic.